

Software Requirements Specification

Ride Booking System

Prepared by:

Youssef Shaher Shoukry

ID: 241015139

Ibrahim Mahmoud Mohamed <Sun 2:30>

ID: 241015503

Aly Eldeen Ahmed

ID: 241016698

Youssef Mohamed Said

ID: 241015841

Youssef Ahmed Mohamed <Tue 2:30>

ID: 241015925

Dr.Ashraf Draz / Engineer Jana

Date

<Monday 10:30>

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience
- 1.4 Project Scope
- 1.5 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Features
- 2.3 User Classes and Characteristics
 - 2.3.1 Rider
 - 2.3.2 Driver
 - 2.3.3 Admin
- 2.4 Operating Environment
- 2.5 Constraints
- 2.6 User Documentation
- 2.7 Assumptions

3. System Features

- 3.1 User Authentication
- 3.2 Ride Request
- 3.3 Ride Management
- 3.4 Payment System

4. External Interface Requirements

- 4.1 User Interface
- 4.2 Hardware Interface
- 4.3 Software Interface
- 4.4 Communication Interface

5. Non-Functional Requirements

- 5.1 Performance
- 5.2 Security
- 5.3 Reliability
- 5.4 Usability

6. System Diagrams

6.1 Use Case Diagram

6.2 Class Diagram

6.3 Sequence Diagram

6.4 Activity Diagram

6.5 State Diagram

1. Introduction

1.1 Purpose

This document defines the software requirements for the **Ride Booking System (RBS)**, a mobile-based application that connects riders with drivers for transportation services.

It provides a detailed description of system functionality, constraints, and expected behavior. This document is intended for developers, testers, and stakeholders to ensure correct system implementation.

1.2 Document Conventions

- **Shall** → Mandatory requirements
- **Should** → Recommended requirements

Abbreviations:

- **FR** = Functional Requirement
 - **NFR** = Non-Functional Requirement
-

1.3 Intended Audience

This document is intended for:

- **Developers** → Implementation
 - **Testers** → Validation
 - **Project Managers** → Planning
 - **Stakeholders** → Review
-

1.4 Project Scope

The Ride Booking System is designed to:

- Allow users to request rides
 - Match riders with nearby drivers
 - Provide real-time trip tracking
-

- Handle secure payments

Out of Scope:

- Vehicle maintenance systems
 - Offline ride booking
-

1.5 References

- IEEE SRS Standards
 - Mobile Application Design Guidelines
-

2. Overall Description

2.1 Product Perspective

The system is a **client-server mobile application** integrated with:

- GPS services
- Payment gateways
- Cloud database

The system integrates with an external **payment gateway** to process card payments securely.

2.2 Product Features

Main features include:

- User registration & login
 - Ride request & matching
 - Real-time tracking
 - Payment processing
 - Admin dashboard
-

2.3 User Classes and Characteristics

Rider

- Basic mobile user
- Requests rides and pays

Driver

- Registered & verified
- Accepts and completes trips

Admin

- Full system access
 - Manages users and trips
-

2.4 Operating Environment

- OS: Android / iOS
 - Backend: Cloud server
 - Database: MySQL / Firebase
 - Internet required
-

2.5 Constraints

- Requires GPS-enabled devices
 - Requires internet connection
 - Must support real-time updates
-

2.6 User Documentation

- User Guide
 - Admin Manual
 - Help Center
-

2.7 Assumptions

- Users have smartphones
 - Drivers are verified
 - Internet is available
-

3. System Features

3.1 Feature: User Authentication

Description & Priority

Allows users (riders & drivers) to securely log in.

Priority: High

Stimulus / Response

Stimulus	Response
User enters valid data	Login successful
Invalid password	Error message

Functional Requirements

- FR-1: User shall be able to register
 - FR-2 User shall be able to login
 - FR-3: System shall validate credentials
-

3.2 Feature: Ride Request

Description & Priority

Allows riders to request a ride.

Priority: High

Stimulus / Response

Stimulus	Response
----------	----------

Rider requests ride	System searches drivers
---------------------	-------------------------

No driver available	Show message
---------------------	--------------

Functional Requirements

- FR-4: User shall be able to requests ride
 - FR-5: User shall accept/reject drivers
 - FR-6: System shall show nearby drivers
 - FR-7: System shall calculate fare
-

3.3 Feature: Ride Management

Description & Priority

Handles trip lifecycle.

Priority: High

Functional Requirements

- FR-8: Driver shall accept / reject rides
 - FR-9: System shall track trip in real time
 - FR-10: System shall mark trip as completed
-

3.4 Feature: Payment System

Description & Priority

The payment system handles fare transactions between riders and drivers using secure payment methods integrated through an external payment gateway.

Priority: High

Functional Requirements

- FR-11: The system shall process payments through an external payment gateway.
 - FR-12: The system shall support multiple payment methods.
 - FR-13: The system shall generate a receipt after each completed transaction.
 - FR-14: The system shall confirm whether a transaction is successful or failed.
 - FR-15: The system shall store payment details associated with each trip.
-

4. External Interface Requirements

4.1 User Interface

- Mobile app screens
 - Dashboard
 - Notifications
-

4.2 Hardware Interface

- GPS sensors
 - Smartphones
-

4.3 Software Interface

- Payment APIs
 - Maps API
 - Database
-

4.4 Communication Interface

- Protocol: HTTPS
 - Format: JSON
 - Real-time updates
-

5. Non-Functional Requirements

5.1 Performance Requirements

- The system should respond to user requests within **3 seconds** under normal conditions.
 - The system should support many concurrent users without performance degradation.
 - The system should efficiently handle real-time ride requests and driver updates.
 - The system should minimize delay in location tracking and trip updates.
-

5.2 Security Requirements

- The system should encrypt user passwords using secure hashing algorithms.
 - The system should ensure secure authentications for all users (Rider, Driver, Admin).
 - The system should use **HTTPS** for all data communication.
 - The system should protect user data from unauthorized access.
 - The system should not store sensitive payment information directly.
-

5.3 Reliability Requirements

- The system should maintain **99.9% uptime**.
 - The system should recover automatically from system failures.
 - The system should ensure data consistency during unexpected shutdowns.
 - The system should handle errors gracefully without crashing.
-

5.4 Usability Requirements

- The system should provide a simple and user-friendly interface.
 - The system should ensure easy navigation across all features.
 - The system should allow users to complete key tasks (e.g., booking a ride) in minimal steps.
 - The system should provide clear messages and feedback to users.
-

5.5 Scalability Requirements

- The system should be scaled to support increasing numbers of users and drivers.
 - The system should handle growth in ride requests without affecting performance.
-

5.6 Availability Requirements

- The system should be available 24/7 for users.
 - The system should minimize downtime during maintenance.
-

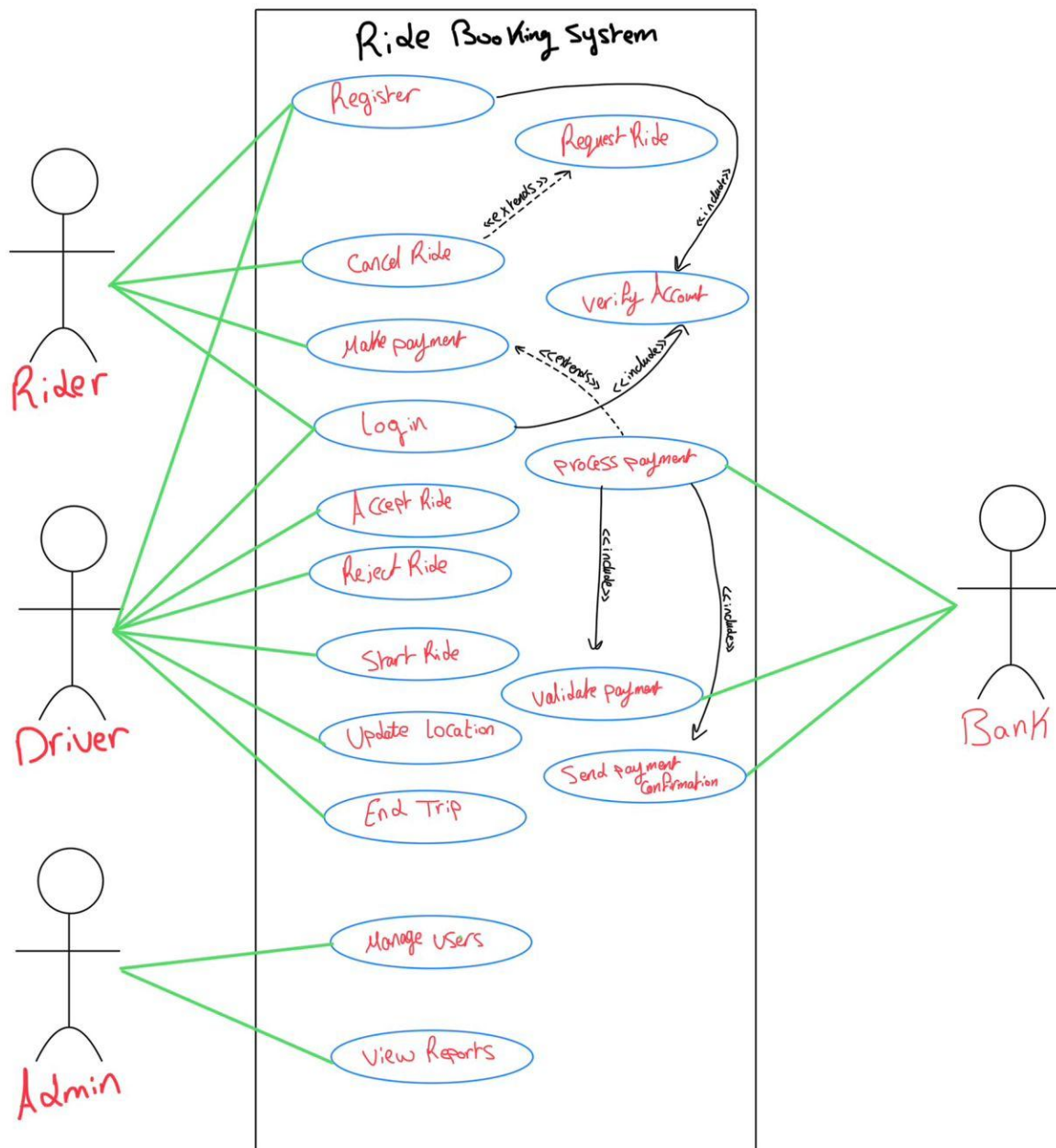
5.7 Maintainability Requirements

- The system should be easy to maintain and update.
 - The system should allow future feature enhancements without major redesign.
-

6. System Diagrams

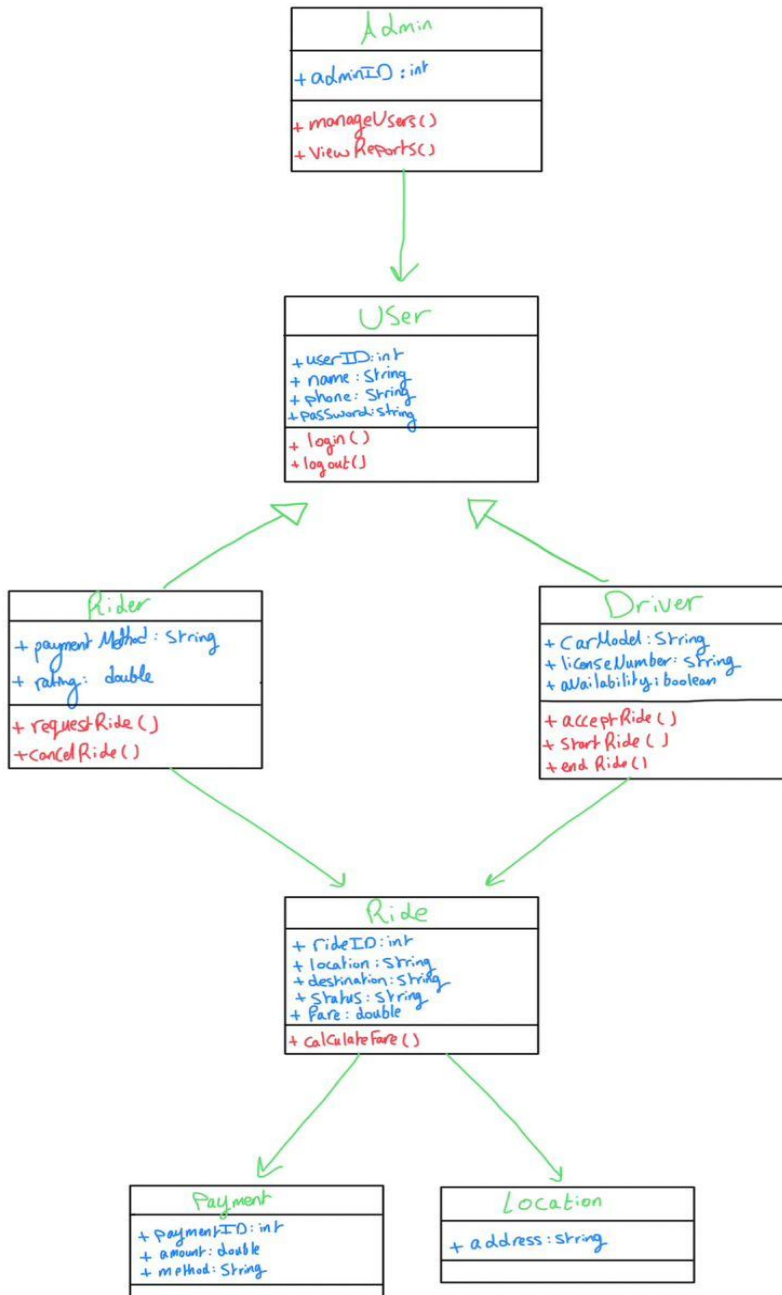
6.1 Use Case Diagram

Use Case Diagram



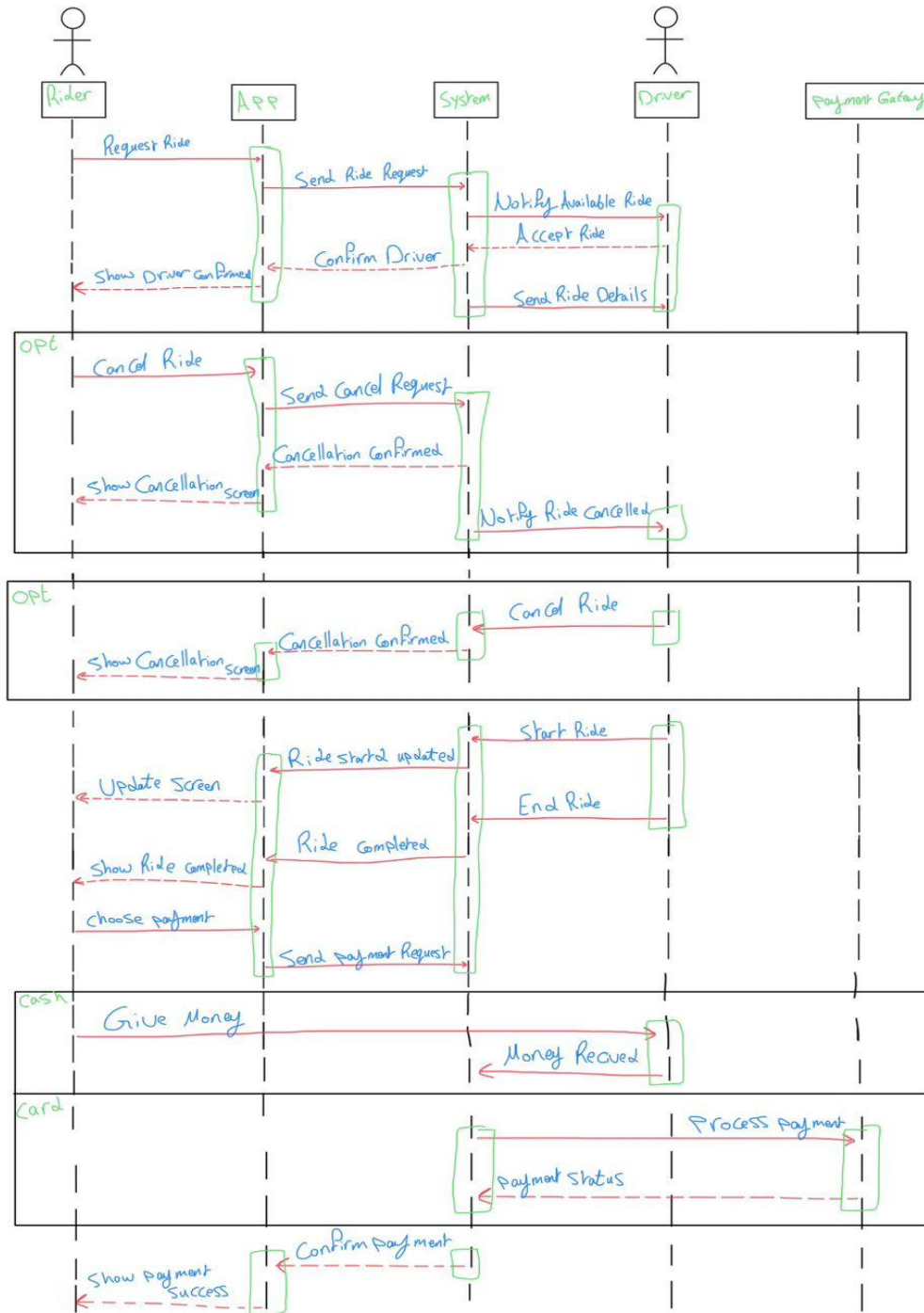
6.2 Class Diagram

class Diagram



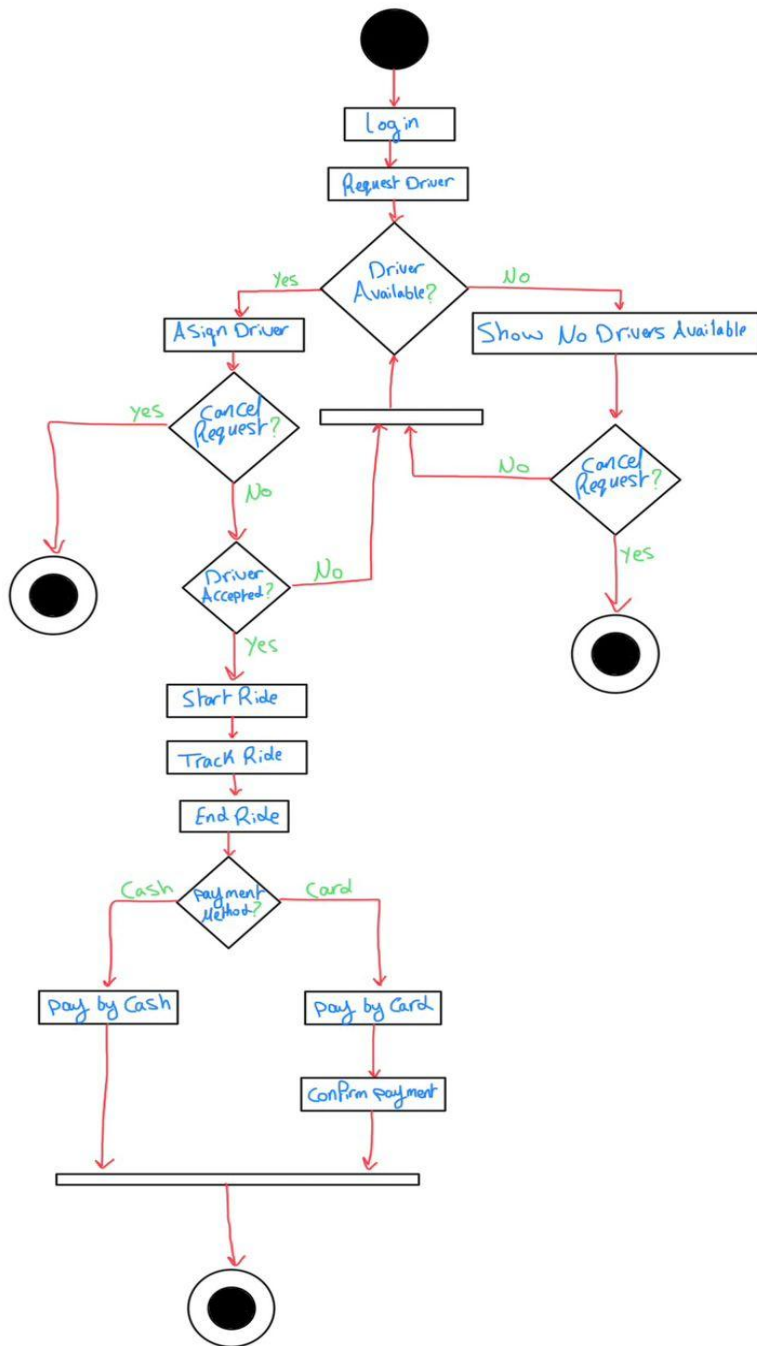
6.3 Sequence Diagram

Sequence Diagram

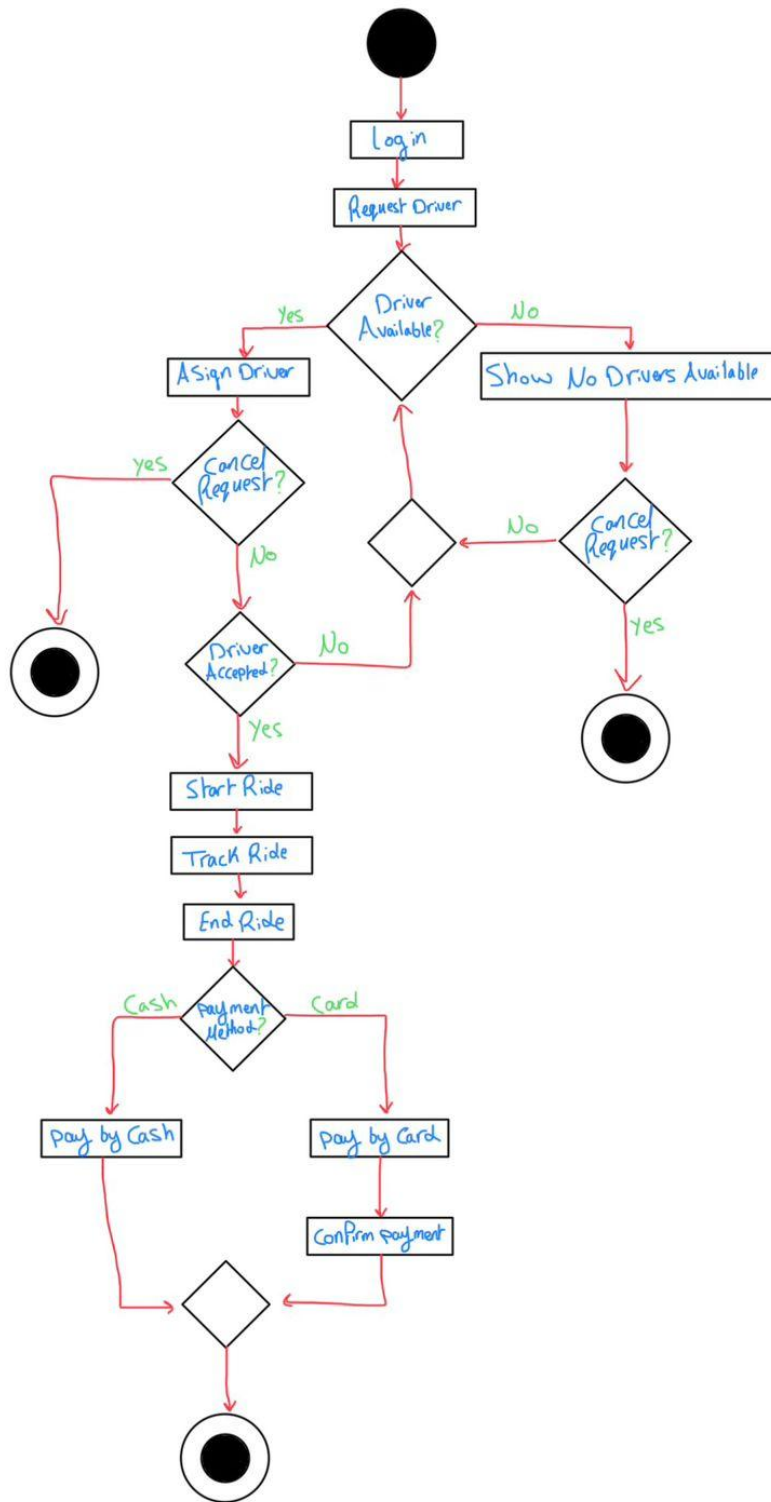


6.4 Activity Diagram

Activity Diagram



Activity Diagram



6.5 State Diagram

State Diagram

